

Arquitectura de Alta Eficiencia para Sistemas Administrativos en Entornos Compartidos: Un Enfoque Integral LAMP

1. Introducción: La Paradoja del Entorno Compartido y la Excelencia Técnica

La concepción de un panel de control "perfecto" en el contexto del desarrollo web moderno a menudo se asocia erróneamente con la adopción de las últimas tecnologías de frontend, como React o Vue.js, acopladas a arquitecturas de microservicios. Sin embargo, cuando el escenario de despliegue se restringe a un **alojamiento compartido (Shared Hosting)**, las reglas del juego cambian drásticamente. En este ecosistema, donde múltiples clientes compiten por ciclos de CPU, memoria RAM limitada y ancho de banda de E/S de disco, la perfección no se define por la novedad tecnológica, sino por la **eficiencia arquitectónica, la robustez de la seguridad y la optimización de recursos**. El desafío técnico radica en construir una aplicación que ofrezca la experiencia de usuario (UX) de una Single Page Application (SPA) moderna, manteniendo al mismo tiempo la ligereza y la facilidad de despliegue de una aplicación PHP tradicional.

El hosting compartido presenta restricciones severas que descalifican muchas arquitecturas modernas. La ausencia de acceso root impide la instalación de demonios persistentes (necesarios para Node.js o WebSockets), la memoria limitada (frecuentemente entre 512MB y 1GB por cuenta) hace arriesgado el uso de frameworks monolíticos pesados que cargan miles de archivos por solicitud, y la falta de acceso a la configuración global del servidor web obliga a depender de archivos .htaccess o .user.ini para ajustes de entorno.¹ Por tanto, la arquitectura ideal para este entorno es aquella que sigue la filosofía "No-Framework" o "Micro-MVC", defendida por figuras como Rasmus Lerdorf, creador de PHP, quien aboga por una arquitectura donde la lógica de negocio y la presentación están desacopladas pero próximas, minimizando el overhead de abstracción.³

Este informe técnico disecciona la construcción de un panel de administración para gestionar usuarios, roles (RBAC) y proyectos, utilizando el stack LAMP (Linux, Apache, MySQL, PHP) con HTML5, CSS3 y JavaScript nativo. El análisis trasciende el código superficial para explorar las decisiones de diseño de segundo y tercer orden necesarias para garantizar seguridad empresarial y rendimiento escalable en un entorno hostil.

2. Ingeniería de la Interfaz de Usuario: Selección y

Optimización del Ecosistema Frontend

La primera capa de interacción en cualquier sistema administrativo es la interfaz de usuario. En un entorno profesional, desarrollar una interfaz desde cero es ineficiente y propenso a errores de usabilidad. La estrategia óptima implica la selección de una plantilla base madura, que actúe como un "framework UI", permitiendo a los desarrolladores centrarse en la lógica de negocio.

2.1 Análisis Comparativo de Soluciones UI: La Supremacía de AdminLTE 3

El mercado de plantillas administrativas está saturado, pero pocas sobreviven al escrutinio técnico riguroso necesario para aplicaciones de larga duración. Al evaluar opciones como **AdminLTE**, **SB Admin 2**, **Stisla** y **CoreUI**, los datos sugieren que **AdminLTE 3** permanece como la opción superior para backends PHP tradicionales, a pesar de la existencia de versiones más nuevas basadas en tecnologías reactivas.⁴

Tabla 1: Evaluación Técnica de Plantillas Administrativas para Entornos PHP

Característica	AdminLTE 3	SB Admin 2	CoreUI (Free)	KeenThemes (Metronic)
Base Tecnológica	Bootstrap 4.6 / jQuery	Bootstrap 4	Bootstrap 5	Bootstrap 5 / Metodología propia
Dependencia de Build Tools	Baja (Funciona sin Node.js)	Media (SCSS recomendado)	Alta (Sass/NPM integrados)	Muy Alta (Gulp/Webpack mandatorio)
Ecosistema de Plugins	Masivo (Select2, DataTables, etc.)	Limitado (Básico)	Limitado en versión Free	Extenso pero complejo
Curva de Integración PHP	Inmediata (Copiar/Pegar assets)	Rápida	Media	Lenta (Requiere compilación)
Estabilidad	Producción	Producción	Activa	Activa

	(LTS)		(Cambios frecuentes)	
--	-------	--	----------------------	--

La elección de AdminLTE 3 se justifica por su dependencia de **jQuery**. Aunque la comunidad de frontend moderna tiende a eliminar jQuery, en el ecosistema PHP y en hosting compartido, jQuery sigue siendo el pegamento universal. La inmensa mayoría de plugins de backend (validación, máscaras de entrada, selectores avanzados) dependen de jQuery. Intentar integrar una plantilla "Vanilla JS" o basada en React en un flujo de trabajo PHP tradicional a menudo resulta en una "reinención de la rueda", donde el desarrollador debe escribir wrappers complejos para funcionalidades que jQuery resuelve en una línea.¹⁰ Además, AdminLTE 3 ofrece una estabilidad visual crítica; sus componentes han sido probados en miles de dispositivos, garantizando que la interfaz sea responsiva y consistente sin necesidad de ajustes CSS manuales constantes.

2.2 Arquitectura del Layout y Experiencia de Usuario (UX)

El diseño "perfecto" debe enmascarar la naturaleza de "recarga de página" de PHP. Aunque no sea una SPA, la percepción de velocidad se logra mediante una arquitectura de layout inteligente y el uso estratégico de AJAX para acciones menores.

La estructura del layout en AdminLTE se compone de cuatro elementos críticos que deben modularizarse en el backend:

1. **Wrapper Principal:** Contenedor fluido que gestiona la responsividad.
2. **Navbar (Barra de Navegación Superior):** Debe configurarse como layout-navbar-fixed.¹² Esto asegura que las herramientas de navegación y perfil de usuario estén siempre accesibles, independientemente de la profundidad del scroll en tablas de datos extensas.
3. **Sidebar (Barra Lateral de Navegación):** Este es el componente más complejo en términos de lógica. Debe ser capaz de renderizar menús anidados y gestionar el estado "activo" basado en la URL actual. Además, su contenido debe filtrarse dinámicamente según los permisos del usuario (RBAC), ocultando módulos a los que el usuario no tiene acceso.⁷
4. **Content Wrapper:** El área de trabajo dinámica. Es imperativo el uso de "Breadcrumbs" (Migas de Pan) automáticas para mantener el contexto del usuario dentro de la jerarquía de la aplicación.

Para optimizar la carga en hosting compartido, se debe evitar la carga monolítica de todos los scripts en el footer. En su lugar, se implementa un sistema de inyección de dependencias de vista. Solo las vistas que requieren gráficos cargan Chart.js; solo las vistas de gestión de usuarios cargan DataTables y SweetAlert. Esta granularidad reduce el peso de la página (Page Size) y acelera el tiempo de renderizado (Time to Interactive), respetando los límites de ancho de banda y conexiones concurrentes del servidor compartido.

3. Arquitectura del Backend: El Paradigma "No-Framework" Optimizado

La decisión más controvertida y crucial en este diseño es la renuncia a frameworks monolíticos como Laravel o Symfony. Aunque potentes, estos frameworks imponen una carga significativa en el sistema de archivos (miles de inodos) y memoria (overhead de arranque del framework) que puede degradar el rendimiento en planes de hosting modestos.¹ El enfoque propuesto es una arquitectura MVC (Modelo-Vista-Controlador) artesanal, optimizada para velocidad de ejecución y simplicidad de mantenimiento.

3.1 Estructura de Directorios y Seguridad por Diseño

La seguridad en hosting compartido comienza con la estructura de directorios. Un error común es colocar toda la aplicación dentro de `public_html`. La arquitectura perfecta separa estrictamente la lógica accesible públicamente de la lógica del núcleo.

Esquema de Directorios Propuesto:

```
/home/usuario/
├── app/ <-- INACCESIBLE VÍA WEB
│   ├── config/ # Credenciales, constantes
│   ├── controllers/ # Orquestadores de lógica
│   ├── core/ # Router, DB Wrapper, Auth
│   ├── helpers/ # Funciones puras
│   ├── models/ # Lógica de datos y negocio
│   └── views/ # Plantillas HTML/PHP
├── logs/ # Logs de errores personalizados
├── public_html/ <-- ACCESIBLE VÍA WEB (WEBROOT)
├── assets/ # CSS, JS, Imágenes (AdminLTE)
├── uploads/ # Archivos de usuario (con.htaccess restrictivo)
├── .htaccess # Front Controller
└── index.php # Punto de entrada único
```

Esta disposición garantiza que, incluso ante un fallo catastrófico del servidor web que exponga el código fuente PHP (un escenario raro pero posible si falla el módulo PHP), los archivos críticos de configuración y modelos de negocio permanecen fuera del alcance de las solicitudes HTTP directas.¹⁴

3.2 El Enrutador y el Patrón Front Controller

El archivo `index.php` actúa como el único punto de entrada, inicializando la sesión, cargando configuraciones y despachando la solicitud. Esto centraliza la seguridad: ninguna página

puede cargarse sin pasar por los filtros de autenticación y configuración global.

El enrutamiento en un entorno sin Composer ni herramientas de consola se maneja mejor mediante la reescritura de URLs en .htaccess y un despachador PHP simple.

Configuración .htaccess para Enrutamiento:

Apache

```
<IfModule mod_rewrite.c>
RewriteEngine On
# Ignorar archivos y directorios reales (imágenes, css)
RewriteCond %{REQUEST_FILENAME}!-f
RewriteCond %{REQUEST_FILENAME}!-d
# Redirigir todo a index.php pasando la ruta como parámetro
RewriteRule ^(.*)$ index.php?route=$1
</IfModule>
```

El despachador PHP (Router) analiza el parámetro route (ej: usuarios/editar/15), lo descompone en Controlador (UserController), Acción (edit) y Parámetros (15). Este enfoque elimina la necesidad de crear un archivo físico PHP para cada página, facilitando URLs limpias y amigables con SEO y usuarios, una práctica estándar que mejora la mantenibilidad a largo plazo.¹⁵

3.3 Gestión de la Base de Datos: Patrón Singleton y PDO

La conexión a la base de datos es el recurso más costoso. En hosting compartido, el límite de conexiones concurrentes ("max_user_connections") suele ser bajo (entre 10 y 50). Abrir múltiples conexiones por solicitud es una receta para el desastre. La implementación del patrón **Singleton** para la clase de base de datos asegura que solo exista una instancia de conexión PDO por ciclo de vida de la solicitud, independientemente de cuántos modelos requieran acceso a datos.

Es imperativo utilizar PDO (PHP Data Objects) en lugar de mysqli debido a su interfaz consistente y soporte para bases de datos nombradas. Una configuración crítica de seguridad en PDO es desactivar la emulación de sentencias preparadas:

```
$pdo->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
```

Esto obliga a que la preparación de la consulta ocurra en el motor de la base de datos MySQL y no en PHP, previniendo vectores avanzados de inyección SQL donde la emulación del lado del cliente podría fallar ante ciertos juegos de caracteres.¹⁸

4. Ingeniería de Datos: Esquema Relacional y Control de Acceso (RBAC)

La solidez de un panel de administración depende de la integridad de sus datos. MySQL con el motor **InnoDB** es obligatorio para soportar transacciones ACID y claves foráneas, garantizando que no queden registros huérfanos (por ejemplo, permisos asignados a un rol que ha sido eliminado).

4.1 Modelo Entidad-Relación para RBAC Jerárquico

Un sistema de permisos simple (campo role en la tabla users) es insuficiente para aplicaciones profesionales. Se requiere un modelo de **Control de Acceso Basado en Roles (RBAC)** normalizado que permita una granularidad fina. El diseño propuesto utiliza un esquema de cinco tablas para máxima flexibilidad.¹⁹

Tabla 2: Esquema de Base de Datos para RBAC

Tabla	Tipo	Descripción y Relaciones
users	Entidad	Almacena credenciales, estado (activo/ban), y perfil.
roles	Entidad	Define agrupaciones funcionales (e.g., 'Administrador', 'Gerente de Proyecto').
permissions	Entidad	Define capacidades atómicas del sistema (e.g., user.create, project.delete).
user_roles	Pivote	Relación N:M entre usuarios y roles. Permite que un usuario

		tenga múltiples roles.
role_permissions	Pivote	Relación N:M entre roles y permisos. Define qué puede hacer cada rol.

Este diseño permite escenarios complejos: un usuario puede ser "Auditor" (solo lectura en finanzas) y "Editor" (escritura en blog) simultáneamente. El sistema verifica la unión de todos los permisos otorgados por sus roles.

4.2 Esquema de Gestión de Proyectos

Para el módulo de proyectos, se necesita una estructura que soporte la colaboración.

- **projects:** Contiene metadatos (id, name, description, start_date, end_date, status, created_by). El campo status debe ser un ENUM o, preferiblemente para escalabilidad, una referencia a una tabla de estados, aunque en sistemas simples un campo indexado (TINYINT) mapeado en PHP es suficiente.
- **project_members:** Tabla pivote que asigna usuarios a proyectos. Es crucial incluir un campo role_in_project en esta tabla si los permisos dentro del proyecto difieren de los roles globales del sistema (e.g., un usuario puede ser "Admin" del sistema pero solo "Observador" en un proyecto específico).

5. Seguridad Avanzada en Capa de Aplicación

En un entorno compartido, no se puede depender de un firewall perimetral (WAF) gestionado por el usuario. La aplicación debe ser autodefensiva.

5.1 Endurecimiento de Sesiones (Session Hardening)

El secuestro de sesiones es una amenaza crítica. PHP, por defecto, puede ser permisivo. La configuración "perfecta" se impone en tiempo de ejecución mediante ini_set al inicio del script index.php.¹⁸

1. **HttpOnly:** ini_set('session.cookie_httponly', 1); impide que scripts maliciosos (XSS) accedan a la cookie de sesión.
2. **Secure:** ini_set('session.cookie_secure', 1); asegura que la cookie solo viaje sobre HTTPS.
3. **Strict Mode:** ini_set('session.use_strict_mode', 1); previene que un atacante fuerce un ID de sesión conocido al usuario (Session Fixation).
4. **Regeneración de ID:** Es vital regenerar el ID de sesión (session_regenerate_id(true))

inmediatamente después del login y periódicamente (e.g., cada 15 minutos) durante la sesión activa para invalidar IDs antiguos que pudieron ser interceptados.

5.2 Protección CSRF (Cross-Site Request Forgery)

Dado que las peticiones administrativas (crear usuario, borrar proyecto) tienen consecuencias graves, se debe implementar un sistema de tokens CSRF. Al no usar frameworks, se debe crear una clase utilitaria `CsrfToken`.

- **Generación:** Al iniciar la sesión, se genera un token criptográficamente seguro (`bin2hex(random_bytes(32))`) y se almacena en `$_SESSION`.
- **Inyección:** Este token se incluye automáticamente como un `<input type="hidden">` en todos los formularios HTML.
- **Verificación:** Un middleware o clase base del controlador intercepta todas las peticiones POST, PUT o DELETE y compara el token recibido con el de la sesión usando `hash_equals()` (para prevenir ataques de tiempo). Si no coinciden, la petición se aborta inmediatamente.¹⁸

5.3 Validación y Saneamiento de Entradas

La confianza cero es el principio rector. Para correos electrónicos, el uso de `filter_var($email, FILTER_VALIDATE_EMAIL)` es mandatorio. Además, para prevenir el spam o registros basura en aplicaciones públicas, se puede implementar una validación contra una lista negra de dominios de correo desechables, verificando el dominio del correo ingresado contra un array de proveedores bloqueados o incluso verificando la existencia de registros DNS MX para ese dominio.²³

Para evitar XSS, toda salida de datos generados por usuarios en las vistas HTML debe ser escapada usando `htmlspecialchars($string, ENT_QUOTES, 'UTF-8')`.

6. Implementación del Módulo de Usuarios: Escalabilidad con DataTables

La gestión de usuarios es el banco de pruebas de rendimiento. Una implementación ingenua que cargue `SELECT * FROM users` y renderice una tabla HTML fallará catastróficamente cuando la base de datos crezca a miles de registros, consumiendo toda la memoria RAM asignada a PHP y bloqueando el navegador del cliente.

6.1 Procesamiento del Lado del Servidor (Server-Side Processing)

La solución definitiva es utilizar el modo **Server-Side** de DataTables. En este modelo, el cliente (navegador) no recibe todos los datos. En su lugar, DataTables actúa como una ventana deslizante, solicitando al servidor solo los registros necesarios para la vista actual

(paginación).²⁶

El flujo de comunicación es el siguiente:

1. **Petición AJAX:** DataTables envía parámetros: draw (contador de secuencia), start (offset SQL), length (límite SQL), search[value] (término de búsqueda global), y order (columnas de ordenamiento).
2. **Lógica SQL Dinámica:** El controlador PHP recibe estos parámetros y construye una consulta SQL optimizada.
 - Aplica filtros WHERE basados en search[value].
 - Aplica ORDER BY dinámico basado en las columnas seleccionadas por el usuario.
 - Aplica LIMIT y OFFSET para devolver solo, por ejemplo, 10 registros.
3. **Respuesta JSON:** El servidor devuelve un objeto JSON con data (el array de 10 usuarios), recordsTotal (total en la tabla) y recordsFiltered (total que coinciden con la búsqueda).

Este enfoque mantiene el consumo de memoria de PHP constante y bajo, independientemente de si hay 100 o 1,000,000 de usuarios en la base de datos, logrando una escalabilidad teórica casi infinita en hosting compartido.²⁹

6.2 Renderizado de Acciones y UX

Para mantener la flexibilidad, la columna de "Acciones" (botones Editar/Eliminar) no se almacena en la base de datos. Se genera en tiempo de ejecución. Existen dos enfoques: generarlo en PHP y enviarlo como string HTML en el JSON, o enviarlo como datos crudos y generarlo en JavaScript mediante `columns.render`. El segundo es preferible arquitectónicamente (separación de intereses), pero el primero es más común en implementaciones rápidas PHP.²⁹

Ejemplo de Definición JS para DataTables:

JavaScript

"columns":

7. Módulo de Proyectos y Lógica de Negocio

El módulo de proyectos requiere una interacción más rica que la simple gestión de usuarios.

Aquí se integran componentes de UI avanzados para mejorar la usabilidad.

7.1 Gestión de Estados con Interruptores (Toggle Switches)

Cambiar el estado de un proyecto (de "Pendiente" a "Activo") no debería requerir abrir un formulario de edición. La implementación perfecta utiliza interruptores **Bootstrap Switch** (nativos en AdminLTE 3) dentro de la tabla de listado.³³

La lógica técnica implica:

1. **Frontend:** Un evento change en el checkbox dispara una petición AJAX asíncrona al controlador ProjectController.
2. **Seguridad:** La petición AJAX debe incluir el token CSRF en los headers.
3. **Backend:** El controlador valida el permiso del usuario (project.update), actualiza el estado en la BD y devuelve un JSON de éxito.
4. **Feedback:** Si la operación es exitosa, se muestra una notificación "Toast" (usando el plugin **SweetAlert2** o **Toastr** incluido en AdminLTE) en la esquina superior derecha. Si falla, el interruptor vuelve a su posición original automáticamente.³⁵

7.2 Asignación de Usuarios (Relaciones Muchos a Muchos)

Asignar usuarios a un proyecto presenta un desafío de interfaz cuando hay miles de usuarios. Un <select> estándar HTML colapsaría el navegador. La solución es utilizar el plugin **Select2** en modo AJAX.

- El usuario escribe en la caja de búsqueda del select.
- Select2 envía la cadena de búsqueda al servidor.
- El servidor busca en la tabla users (usando índices LIKE 'term%') y devuelve resultados paginados.
- El usuario selecciona múltiples colaboradores.

Al guardar el proyecto, el backend recibe un array de IDs de usuarios. La lógica de negocio debe gestionar la sincronización de la tabla project_members: eliminar asociaciones que ya no existen en el envío y crear las nuevas, todo dentro de una **transacción de base de datos** para asegurar la integridad de los datos.³⁶

8. Despliegue, Mantenimiento y Estrategias de Backup

El ciclo de vida del software no termina con el código. En hosting compartido, el despliegue requiere disciplina.

8.1 Control de Versiones y Despliegue FTP

Aunque el acceso SSH y Git pueden no estar disponibles, el desarrollo debe realizarse localmente utilizando Git. Para el despliegue, herramientas como **Git-FTP** o pipelines de

CI/CD (GitHub Actions) pueden configurarse para subir solo los archivos modificados vía FTP, evitando la propensión a errores de las subidas manuales con clientes FTP gráficos.

Es crucial excluir del despliegue archivos de desarrollo como .git, README.md, composer.json (si se usó localmente para autoloader), y archivos de configuración local. El archivo config.php en producción debe ser creado manualmente una única vez con las credenciales de producción y permisos de archivo 600 o 644.

8.2 Gestión de Errores y Logs

En producción, la directiva display_errors debe estar estrictamente en Off. Mostrar errores PHP al usuario final es una vulnerabilidad de divulgación de información (Full Path Disclosure). Se debe configurar un manejador de errores personalizado (set_error_handler) que capture advertencias y excepciones y las escriba en un archivo de log rotativo dentro del directorio privado /app/logs/error.log. Esto permite a los administradores depurar problemas sin exponer detalles técnicos al público.¹⁸

8.3 Automatización de Backups

Aunque el proveedor de hosting suele ofrecer backups, la aplicación debe ser responsable de sus datos. Se puede implementar un script PHP disparado por un **Cron Job** (disponible en cPanel) que realice un mysqldump de la base de datos, comprima el archivo (gzip) y lo guarde en una carpeta segura o lo envíe a un almacenamiento externo (como AWS S3 o vía FTP a otro servidor), asegurando la continuidad del negocio ante fallos catastróficos del proveedor.

9. Conclusión

La arquitectura detallada en este informe demuestra que es posible construir un sistema administrativo de clase mundial en un entorno de recursos restringidos. Al combinar la estabilidad visual de **AdminLTE 3**, la eficiencia de un **MVC PHP nativo**, la potencia de **DataTables Server-Side**, y un modelo de seguridad **RBAC** riguroso, se obtiene un "Panel de Control Perfecto": rápido, seguro, escalable y mantenible. Esta solución evita la deuda técnica de frameworks sobredimensionados en infraestructura pequeña, alineando la tecnología con la realidad operativa del despliegue.

Obras citadas

1. Comparing The Best PHP Frameworks for Shared Hosting | FastComet, fecha de acceso: enero 7, 2026, <https://www.fastcomet.com/blog/best-php-frameworks-shared-hosting>
2. how to run laravel 5 in shared hosting without composer? - Stack Overflow, fecha de acceso: enero 7, 2026, <https://stackoverflow.com/questions/40505726/how-to-run-laravel-5-in-shared-hosting-without-composer>

3. The no-framework PHP MVC framework | Toys, fecha de acceso: enero 7, 2026, <https://toys.lerdorf.com/the-no-framework-php-mvc-framework>
4. AdminKit: Free Bootstrap 5 Admin Template, fecha de acceso: enero 7, 2026, <https://adminkit.io/>
5. 50+ Free Bootstrap 4 Admin Dashboard Templates 2024, fecha de acceso: enero 7, 2026, <https://cssauthor.com/free-bootstrap-4-admin-dashboard-templates/>
6. Turn AdminLTE, SB Admin, Tabler & Any Bootstrap Template Into a Working Application, fecha de acceso: enero 7, 2026, <https://www.gridphp.com/datagrid-with-admin-templates/>
7. Upgrade Guide | AdminLTE 3 Documentation, fecha de acceso: enero 7, 2026, <https://adminlte.io/docs/3.0/upgrade-guide.html>
8. Free Bootstrap Admin Template · Open Source Dashboard & Panel - CoreUI, fecha de acceso: enero 7, 2026, <https://coreui.io/product/free-bootstrap-admin-template/>
9. Tailwind, Bootstrap & React Templates, fecha de acceso: enero 7, 2026, <https://keenthemes.com/>
10. Top 10+ Simple Admin Themes and Templates - Flatlogic Blog, fecha de acceso: enero 7, 2026, <https://flatlogic.com/blog/top-10-simple-admin-themes-and-templates/>
11. AdminLTE - Free admin dashboard template based on Bootstrap 5 - GitHub, fecha de acceso: enero 7, 2026, <https://github.com/ColorlibHQ/AdminLTE>
12. Layout | AdminLTE v3 Documentation, fecha de acceso: enero 7, 2026, <https://adminlte.io/docs/3.1/layout.html>
13. Push Menu Plugin | AdminLTE 3 Documentation, fecha de acceso: enero 7, 2026, <https://adminlte.io/docs/3.0/javascript/push-menu.html>
14. Web site creation in MVC, and with out any frameworks, but in ..., fecha de acceso: enero 7, 2026, <https://stackoverflow.com/questions/1443141/web-site-creation-in-mvc-and-with-out-any-frameworks-but-in-simple-php-what-w>
15. Building a PHP Site Without a Framework | by Shuqi Khor - Medium, fecha de acceso: enero 7, 2026, <https://sqkhor.medium.com/building-a-php-site-without-a-framework-dca731f42ab>
16. Fast PHP Routing with PHRoute - SitePoint, fecha de acceso: enero 7, 2026, <https://www.sitepoint.com/fast-php-routing-phroute/>
17. Building a Basic Router with PHP: A Step-by-Step Guide for Web Developers | Jotform Tech, fecha de acceso: enero 7, 2026, <https://tech.jotform.com/what-is-router-and-how-to-create-your-own-router-with-php-fad811cf2873>
18. Best Practices for Secure PHP Coding - DEV Community, fecha de acceso: enero 7, 2026, <https://dev.to/patoliyainfotech/best-practices-for-secure-php-coding-im2>
19. Role-Based Access Control Best Practices: 11 Top Tips - Cerbos, fecha de acceso: enero 7, 2026, <https://www.cerbos.dev/blog/role-based-access-control-best-practices>

20. Implementation of Role-Based Access Control in PHP - GitHub, fecha de acceso: enero 7, 2026, <https://github.com/hamza1886/role-based-access-control>
21. 10 Session Management Security Best Practices - Endgrate, fecha de acceso: enero 7, 2026, <https://endgrate.com/blog/10-session-management-security-best-practices>
22. How PHP Sessions Help Maintain State in Web Applications - Test-king.com, fecha de acceso: enero 7, 2026, <https://www.test-king.com/blog/how-php-sessions-help-maintain-state-in-web-applications/>
23. 3 Methods to Validate Emails in PHP: Code Snippets Included - MailerSend, fecha de acceso: enero 7, 2026, <https://www.mailersend.com/blog/php-email-validation>
24. How to validate email by whitelist domain regex - Stack Overflow, fecha de acceso: enero 7, 2026, <https://stackoverflow.com/questions/54087566/how-to-validate-email-by-whitelists-domain-regex>
25. Block any registration that does not use @gmail as mail provider - Ultimate Member, fecha de acceso: enero 7, 2026, <https://docs.ultimatemember.com/article/62-block-any-registration-that-does-not-use-gmail-as-mail-provider>
26. DataTables server-side processing example - Funceme, fecha de acceso: enero 7, 2026, http://funceme.br/app/js/ext/DataTables-1.9.4/examples/data_sources/server_side.html
27. Server-side processing - DataTables, fecha de acceso: enero 7, 2026, <https://datatables.net/manual/server-side>
28. DataTables example - Server-side processing, fecha de acceso: enero 7, 2026, https://datatables.net/examples/data_sources/server_side.html
29. DataTables example - Server-side processing, fecha de acceso: enero 7, 2026, https://datatables.net/examples/server_side/simple.html
30. DataTables server-side processing example, fecha de acceso: enero 7, 2026, https://exacq.com/support/dtjq/examples/server_side/server_side.html
31. DataTables Server-side Processing with Custom Links and Buttons using PHP, fecha de acceso: enero 7, 2026, <https://www.codexworld.com/datatables-server-side-processing-add-links-buttons-to-row-column-with-php/>
32. How do I add button on each row in datatable? - Stack Overflow, fecha de acceso: enero 7, 2026, <https://stackoverflow.com/questions/22471862/how-do-i-add-button-on-each-row-in-datatable>
33. toggle-switch.blade.php - InfyOmLabs/adminlte-templates - GitHub, fecha de acceso: enero 7, 2026, <https://github.com/InfyOmLabs/adminlte-templates/blob/develop/views/templates/fields/toggle-switch.blade.php>
34. General Form Elements - AdminLTE 3, fecha de acceso: enero 7, 2026, <https://adminlte.io/themes/v3/pages/forms/general.html>

35. Modals & Alerts - AdminLTE 3, fecha de acceso: enero 7, 2026,
<https://adminlte.io/themes/v3/pages/UI/modals.html>
36. Laravel AdminLTE download | SourceForge.net, fecha de acceso: enero 7, 2026,
<https://sourceforge.net/projects/laravel-adminlte.mirror/>